Resultados de la práctica

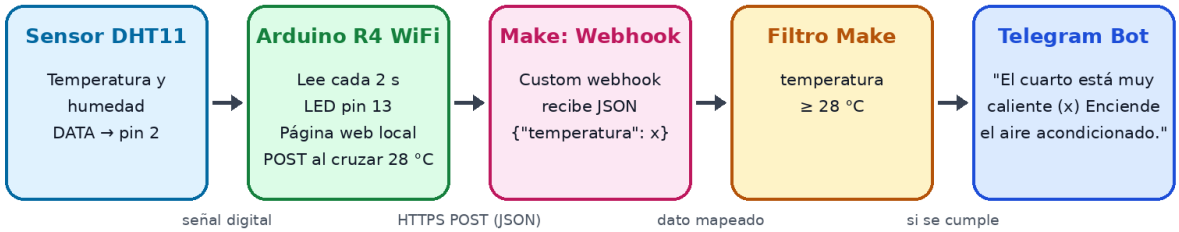
Arduino_Leer_DTH11_Make_Telegram

Dato	Valor
Alumno	Christian Paul Lizarraga Oronia
Carrera	Ingeniería en Sistemas Computacionales
Profesor	Miguel Ángel Barrón Hernández
Fecha de la prueba	23 de septiembre de 2026

1. ¿Qué se hizo?

Se construyó un sistema de alerta por temperatura. Un **Arduino UNO R4 WiFi** lee un sensor **DHT11** cada 2 segundos; cuando la temperatura llega a **28 °C** enciende el LED del pin 13 y manda un POST en JSON a un **Webhook de Make.com**. El escenario "Leer DTH11" pasa el dato por un **filtro** (temperatura ≥ 28 °C) y el **Telegram Bot** envía el aviso "El cuarto está muy caliente (x) Enciende el aire acondicionado." El Arduino también muestra los datos en una página web local que se actualiza cada 5 segundos.

Flujo de la práctica "Leer DTH11"



Conexiones

DHT11	Arduino UNO R4 WiFi
VCC	5V
GND	GND
DATA	Pin digital 2 (pull-up 10 kΩ si el módulo no la trae)
LED indicador	Pin 13

Figura 1. Flujo del sistema y conexiones del sensor.

2. Resultados obtenidos

El escenario quedó activo ("Immediately as data arrives") y el sistema funcionó de punta a punta. Después de enviar */start* al bot (19:58), llegaron estas alertas:

Hora	Temperatura	¿Se envió la alerta?	Observación
20:03	29.5 °C	Sí	Primer cruce del umbral; el mensaje trae el valor real.
20:37	28 °C	Sí	La temperatura bajó del umbral y volvió a subir.
20:39	28 °C	Sí	Nueva oscilación justo en el límite de 28 °C.

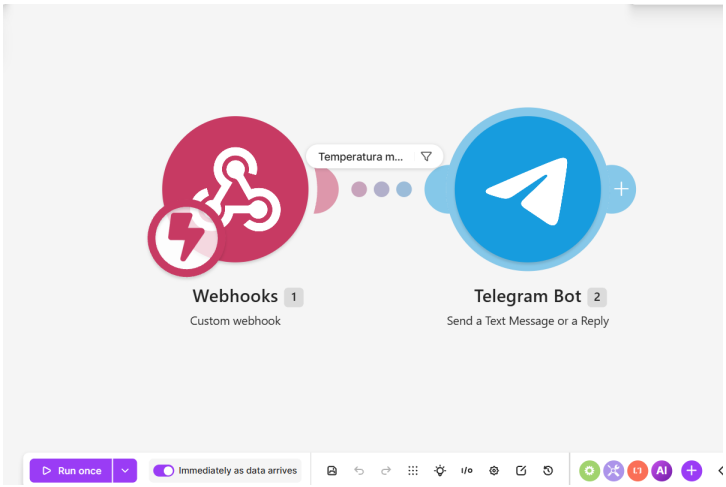


Figura 2. Escenario "Leer DTH11" en Make: Webhooks → filtro → Telegram Bot.

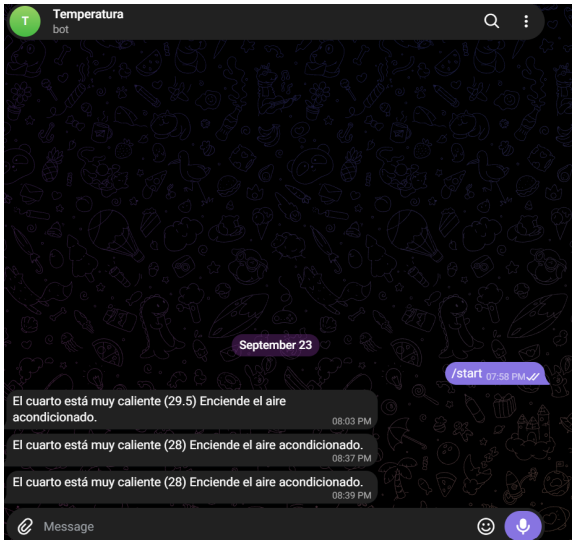


Figura 3. Alertas recibidas en el chat del bot "Temperatura".

3. Lo que aprendí

- **Conectar hardware con la nube sin servidor propio.** El Arduino funciona como cliente HTTPS que solo "llama" cuando hay algo que reportar; Make espera en la URL del Webhook y hace el resto.
- **Trabajar con JSON y Webhooks.** El nombre del campo que manda el Arduino (*temperatura*) debe ser exactamente el que detecta Make; si no, el dato no se puede mapear en Telegram.
- **Detectar cambios de estado en lugar de repetir.** Con la variable *estadoCaliente* el Arduino solo avisa en el momento en que se cruza el umbral, lo que evita llenar el chat de mensajes y gastar operaciones de Make.
- **Validar en dos lugares.** El filtro de Make (≥ 28 °C) es una segunda revisión que evita alertas falsas y deja la regla visible y fácil de cambiar sin reprogramar la placa.
- **Límites del sensor.** El DHT11 tiene una precisión de ±2 °C y necesita al menos 2 segundos entre lecturas. Cerca del umbral la lectura "brinca" entre 27 y 28 °C, y eso explica las alertas repetidas de las 20:37 y las 20:39.
- **Requisitos prácticos.** El UNO R4 WiFi solo se conecta a redes de 2.4 GHz, el bot necesita un */start* del usuario antes de poder escribirle, y el escenario debe estar activo para reaccionar en tiempo real.
- **Seguridad.** La contraseña del WiFi y la URL del Webhook no deben subirse a un repositorio público, porque con la URL cualquiera podría enviar mensajes al bot.

4. Relación con el desarrollo sustentable

La práctica muestra cómo la automatización ayuda a usar mejor la energía: el aire acondicionado se enciende solo cuando la temperatura lo justifica, en lugar de dejarlo prendido por costumbre. El sistema también aplica ideas de **Green IT**: usa una placa de muy bajo consumo y trabaja por eventos (solo se comunica cuando algo cambia), así que genera poco tráfico de red y pocas ejecuciones en la nube. Llevado a un salón o a una oficina, el mismo esquema puede ayudar a reducir el consumo eléctrico y las emisiones que produce.

5. Mejoras propuestas

- Agregar **histéresis** (avisar a los 28 °C y rearmar la alerta al bajar de 27 °C) para evitar mensajes repetidos en el límite.
- Enviar un segundo aviso cuando la temperatura vuelva a la normalidad para apagar el aire.
- Registrar las lecturas en Google Sheets desde Make para tener un historial y graficarlo.
- Usar un DHT22 (más preciso) o incluir la humedad y el índice de calor en el mensaje.

6. Conclusión

Se cumplió el objetivo: el sensor detectó la temperatura alta, el Arduino avisó a Make y el mensaje llegó a Telegram con el valor real, sin intervención manual. Lo más valioso fue entender que un sistema IoT depende de que todas sus partes coincidan (URL, formato del dato y estado del escenario) y de diseñarlo para que se comunique solo cuando hace falta. Así se ahorran recursos, que es justamente la idea detrás de la tecnología sustentable.